

✓ TASK 2 – DATA VISUALIZATION DASHBOARD

Sales Performance Analysis

Objective

The objective of this project is to analyze sales data and create meaningful visualizations using Python libraries such as Pandas, Matplotlib, Seaborn, and Plotly.

Visualizations Created

- Bar Chart – Sales by Category
- Line Chart – Monthly Sales Trend
- Pie Chart – Sales Distribution by Region
- Histogram – Sales Distribution
- Scatter Plot – Sales vs Profit

Tools Used

- Google Colab
- Python
- Pandas
- Matplotlib
- Seaborn
- Plotly

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

%matplotlib inline

sns.set_theme(style="whitegrid")

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
from google.colab import files
```

```
uploaded = files.upload()
```

[Choose Files](#) Sample-Superstore.csv
Sample-Superstore.csv(text/csv) - 2477201 bytes, last modified: 19/9/2026 - 100% done
Saving Sample-Superstore.csv to Sample-Superstore.csv

```
import pandas as pd

df = pd.read_csv("Sample-Superstore.csv")

df.head()
```

	csvbase_row_id	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	...	Postal Code	Region	Product ID	Cate
0	1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	...	42420	South	FUR-BO-10001798	Furr
1	2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	...	42420	South	FUR-CH-10000454	Furr
2	3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	...	90036	West	OFF-LA-10000240	C Sup
3	4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	...	33311	South	FUR-TA-10000577	Furr
4	5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	...	33311	South	OFF-ST-10000760	C Sup

5 rows × 22 columns

```
# Check the number of rows and columns
print("Dataset Shape:")
print(df.shape)
```

```
# Check column names
print("\nColumn Names:")
print(df.columns.tolist())
```

```
# Check data types and missing values
print("\nDataset Information:")
df.info()
```

Dataset Shape:
(9994, 22)

Column Names:
['csvbase_row_id', '\uffeffRow ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment'

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 22 columns):

#	Column	Non-Null Count	Dtype
0	csvbase_row_id	9994 non-null	int64
1	Row ID	9994 non-null	int64
2	Order ID	9994 non-null	object
3	Order Date	9994 non-null	object
4	Ship Date	9994 non-null	object
5	Ship Mode	9994 non-null	object
6	Customer ID	9994 non-null	object
7	Customer Name	9994 non-null	object
8	Segment	9994 non-null	object
9	Country	9994 non-null	object
10	City	9994 non-null	object
11	State	9994 non-null	object
12	Postal Code	9994 non-null	int64
13	Region	9994 non-null	object
14	Product ID	9994 non-null	object
15	Category	9994 non-null	object
16	Sub-Category	9994 non-null	object
17	Product Name	9994 non-null	object
18	Sales	9994 non-null	float64
19	Quantity	9994 non-null	int64
20	Discount	9994 non-null	float64
21	Profit	9994 non-null	float64

dtypes: float64(3), int64(4), object(15)

memory usage: 1.7+ MB

```
# Check missing values
print("Missing Values in Each Column:")
print(df.isnull().sum())
```

```
Missing Values in Each Column:
csvbase_row_id    0
Row ID            0
Order ID          0
Order Date        0
Ship Date         0
Ship Mode         0
Customer ID       0
Customer Name     0
Segment          0
Country           0
City              0
State             0
Postal Code       0
Region           0
Product ID        0
Category          0
Sub-Category      0
Product Name      0
Sales             0
Quantity          0
Discount          0
Profit            0
dtype: int64
```

```
# Check duplicate rows
print("Number of duplicate rows:", df.duplicated().sum())
```

```
Number of duplicate rows: 0
```

```
# Convert date columns to datetime format
```

```
df["Order Date"] = pd.to_datetime(df["Order Date"])
df["Ship Date"] = pd.to_datetime(df["Ship Date"])
```

```
# Check the data types
print(df[["Order Date", "Ship Date"]].dtypes)
```

```
Order Date    datetime64[ns]
Ship Date     datetime64[ns]
dtype: object
```

```
# Create Year and Month columns for analysis
```

```
df["Year"] = df["Order Date"].dt.year
df["Month"] = df["Order Date"].dt.month
df["Month Name"] = df["Order Date"].dt.month_name()
```

```
# Display the new columns
df[["Order Date", "Year", "Month", "Month Name"]].head()
```

	Order Date	Year	Month	Month Name
0	2016-11-08	2016	11	November
1	2016-11-08	2016	11	November
2	2016-06-12	2016	6	June
3	2015-10-11	2015	10	October
4	2015-10-11	2015	10	October

```
# Calculate profit margin
```

```
df["Profit Margin"] = (df["Profit"] / df["Sales"]) * 100
```

```
df[["Sales", "Profit", "Profit Margin"]].head()
```

	Sales	Profit	Profit Margin	
0	261.9600	41.9136	16.00	
1	731.9400	219.5820	30.00	
2	14.6200	6.8714	47.00	
3	957.5775	-383.0310	-40.00	
4	22.3680	2.5164	11.25	

```
# Display the final dataset information
```

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
df.head()
```

```
Number of rows: 9994
```

```
Number of columns: 26
```

	csvbase_row_id	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	...	Sub-Category	Product Name	Sales
0	1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	...	Bookcases	Bush Somerset Collection Bookcase	261.9600
1	2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	...	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400
2	3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	...	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200
3	4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	...	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775
4	5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	...	Storage	Eldon Fold 'N Roll Cart System	22.3680

```
5 rows x 26 columns
```

```
# =====
# BAR CHART - SALES BY CATEGORY
# =====

category_sales = df.groupby("Category")["Sales"].sum().sort_values(ascending=False)

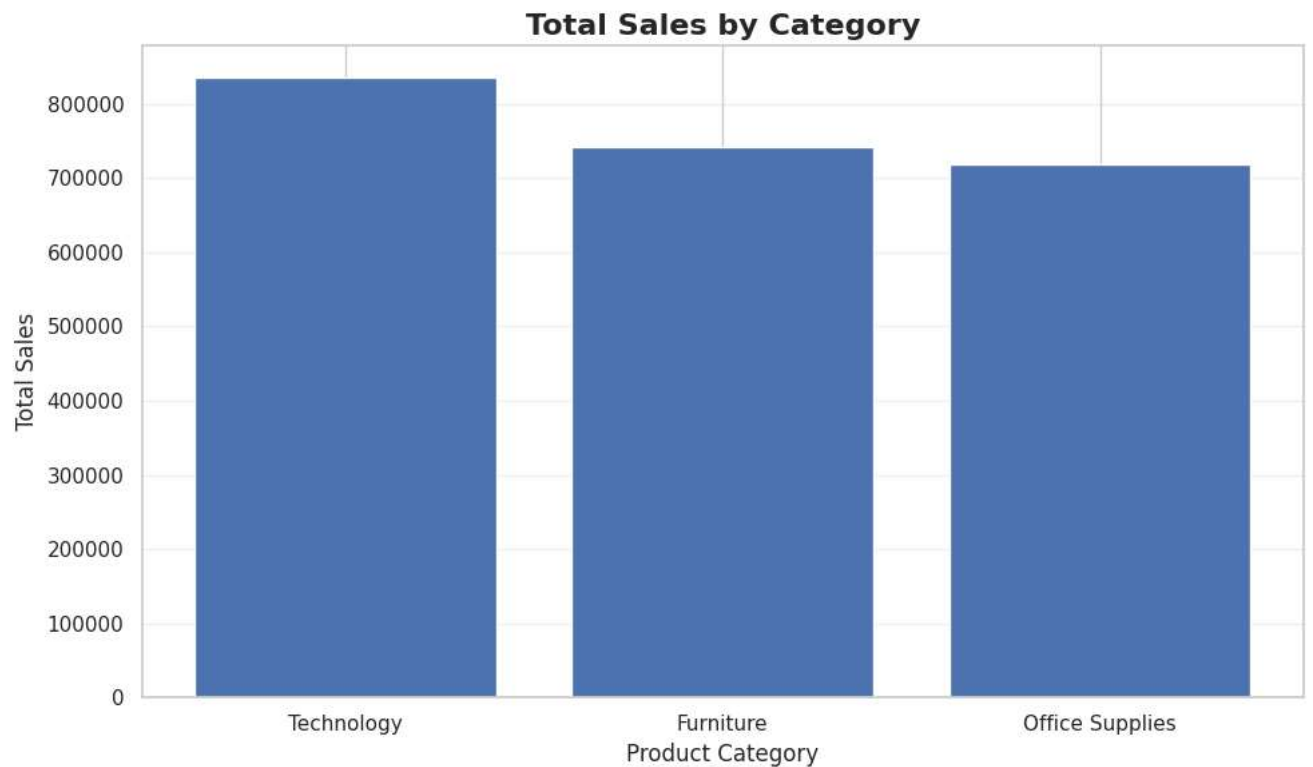
plt.figure(figsize=(10, 6))

plt.bar(
    category_sales.index,
    category_sales.values
)

plt.title("Total Sales by Category", fontsize=16, fontweight="bold")
plt.xlabel("Product Category", fontsize=12)
plt.ylabel("Total Sales", fontsize=12)

plt.xticks(rotation=0)
plt.grid(axis="y", alpha=0.3)

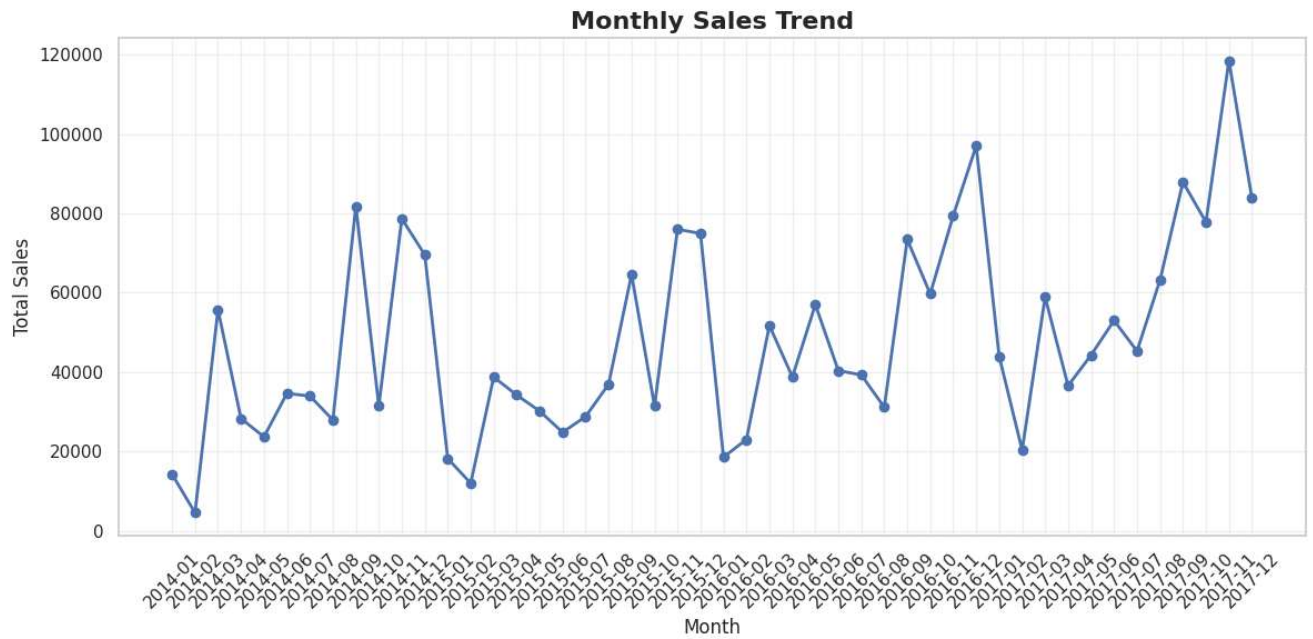
plt.tight_layout()
plt.show()
```



✖ Insight

The bar chart compares the total sales generated by each product category. It helps identify which categories contribute the most to overall sales and which categories have relatively lower sales.

```
# =====  
# LINE CHART - MONTHLY SALES TREND  
# =====  
  
monthly_sales = (  
    df.groupby(df["Order Date"].dt.to_period("M"))["Sales"]  
    .sum()  
)  
  
monthly_sales.index = monthly_sales.index.astype(str)  
  
plt.figure(figsize=(12, 6))  
  
plt.plot(  
    monthly_sales.index,  
    monthly_sales.values,  
    marker="o",  
    linewidth=2  
)  
  
plt.title("Monthly Sales Trend", fontsize=16, fontweight="bold")  
plt.xlabel("Month", fontsize=12)  
plt.ylabel("Total Sales", fontsize=12)  
  
plt.xticks(rotation=45)  
plt.grid(True, alpha=0.3)  
  
plt.tight_layout()  
plt.show()
```



```
df["Order Date"].dt.to_period("M")
```

	Order Date
0	2016-11
1	2016-11
2	2016-06
3	2015-10
4	2015-10
...	...
9989	2014-01
9990	2017-02
9991	2017-02
9992	2017-02
9993	2017-05

9994 rows × 1 columns

dtype: period[M]

```
# =====
# 1. BAR CHART - SALES BY CATEGORY
# =====

category_sales = df.groupby("Category")["Sales"].sum().sort_values(ascending=False)

plt.figure(figsize=(10, 6))

bars = plt.bar(category_sales.index, category_sales.values)

plt.title("Total Sales by Category", fontsize=16, fontweight="bold")
plt.xlabel("Product Category", fontsize=12)
plt.ylabel("Total Sales", fontsize=12)

plt.grid(axis="y", alpha=0.3)

# Add values above bars
```

```

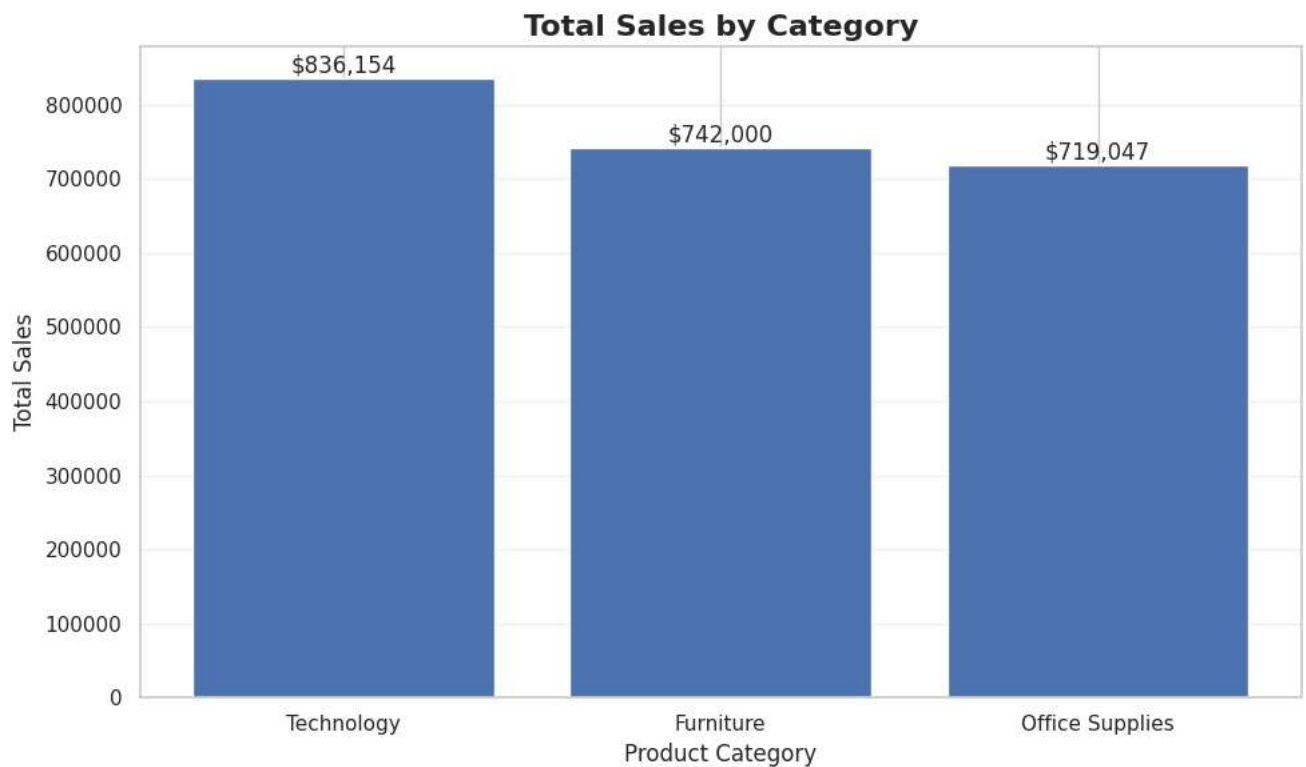
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
        height,
        f"${height:,.0f}",
        ha="center",
        va="bottom"
    )

```

```

plt.tight_layout()
plt.show()

```



```

# =====
# 3. PIE CHART - SALES BY REGION
# =====

region_sales = df.groupby("Region")["Sales"].sum().sort_values(ascending=False)

plt.figure(figsize=(8, 8))

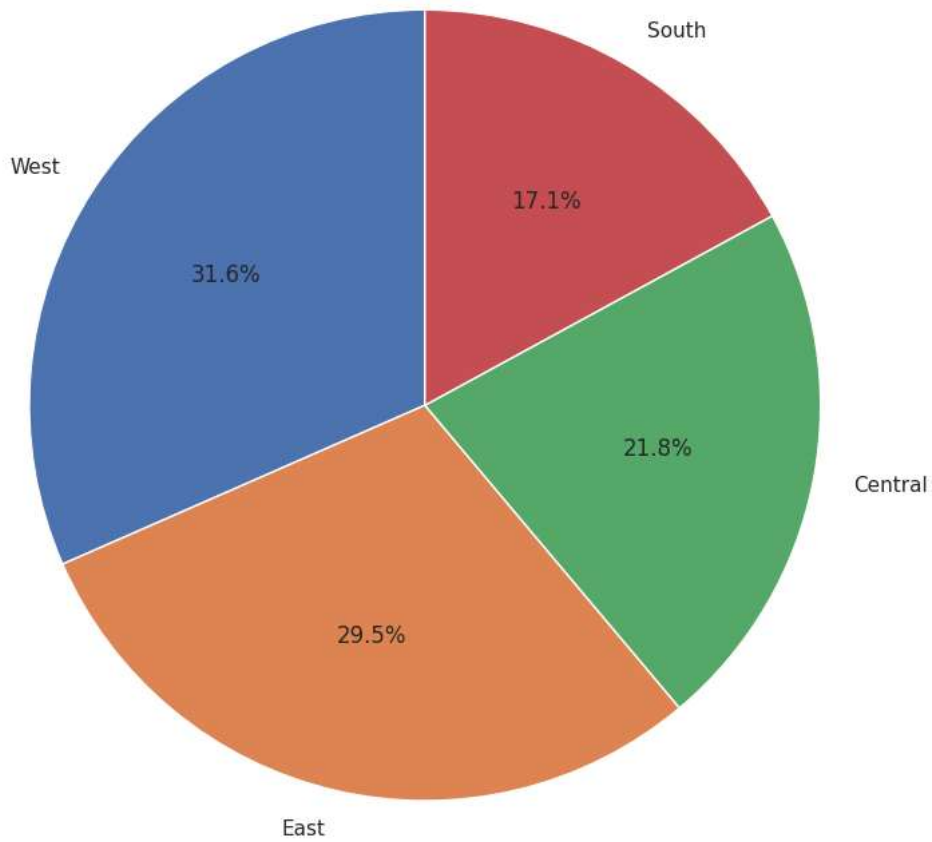
plt.pie(
    region_sales.values,
    labels=region_sales.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title(
    "Sales Distribution by Region",
    fontsize=16,
    fontweight="bold"
)

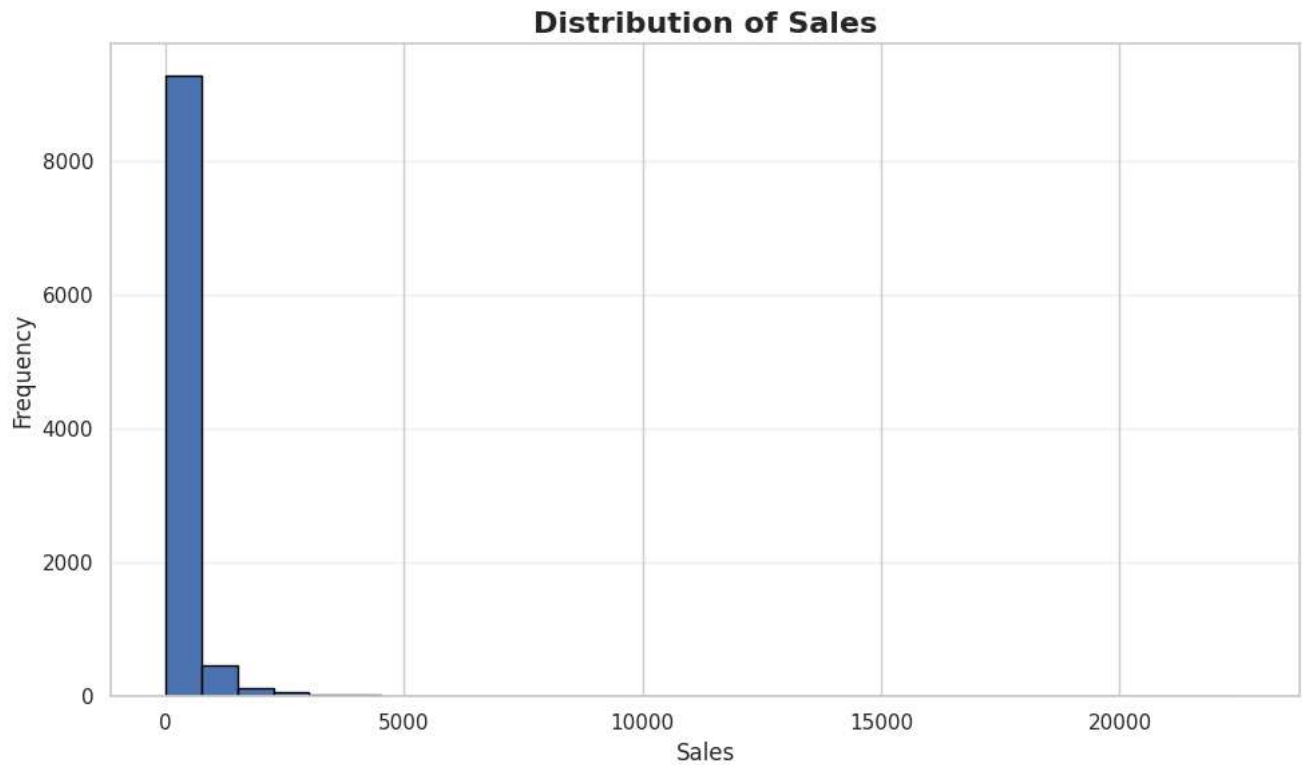
plt.tight_layout()
plt.show()

```

Sales Distribution by Region



```
# =====  
# 4. HISTOGRAM - SALES DISTRIBUTION  
# =====  
  
plt.figure(figsize=(10, 6))  
  
plt.hist(  
    df["Sales"],  
    bins=30,  
    edgecolor="black"  
)  
  
plt.title(  
    "Distribution of Sales",  
    fontsize=16,  
    fontweight="bold"  
)  
  
plt.xlabel("Sales", fontsize=12)  
plt.ylabel("Frequency", fontsize=12)  
  
plt.grid(axis="y", alpha=0.3)  
  
plt.tight_layout()  
plt.show()
```

▼ Insight

The histogram shows the distribution of individual sales values. It helps identify the most common sales ranges and reveals whether the data contains very high or very low sales values.

```
# =====  
# 5. SCATTER PLOT - SALES VS PROFIT  
# =====  
  
plt.figure(figsize=(10, 6))  
  
plt.scatter(  
    df["Sales"],  
    df["Profit"],  
    alpha=0.5  
)  
  
plt.title(  
    "Sales vs Profit",  
    fontsize=16,  
    fontweight="bold"  
)  
  
plt.xlabel("Sales", fontsize=12)  
plt.ylabel("Profit", fontsize=12)  
  
plt.axhline(y=0, linestyle="--", linewidth=1)  
  
plt.grid(True, alpha=0.3)  
  
plt.tight_layout()  
plt.show()
```



Insight

The scatter plot examines the relationship between sales and profit. The distribution of points helps identify whether higher sales are generally associated with higher profit and also highlights transactions with low or negative profit.

Key Performance Indicators

The following metrics provide a high-level summary of the business performance.

```
# =====
# KEY PERFORMANCE INDICATORS
# =====

total_sales = df["Sales"].sum()
total_profit = df["Profit"].sum()
total_quantity = df["Quantity"].sum()
total_orders = df["Order ID"].nunique()
average_sales = df["Sales"].mean()

print("=" * 45)
print("      BUSINESS PERFORMANCE")
print("=" * 45)

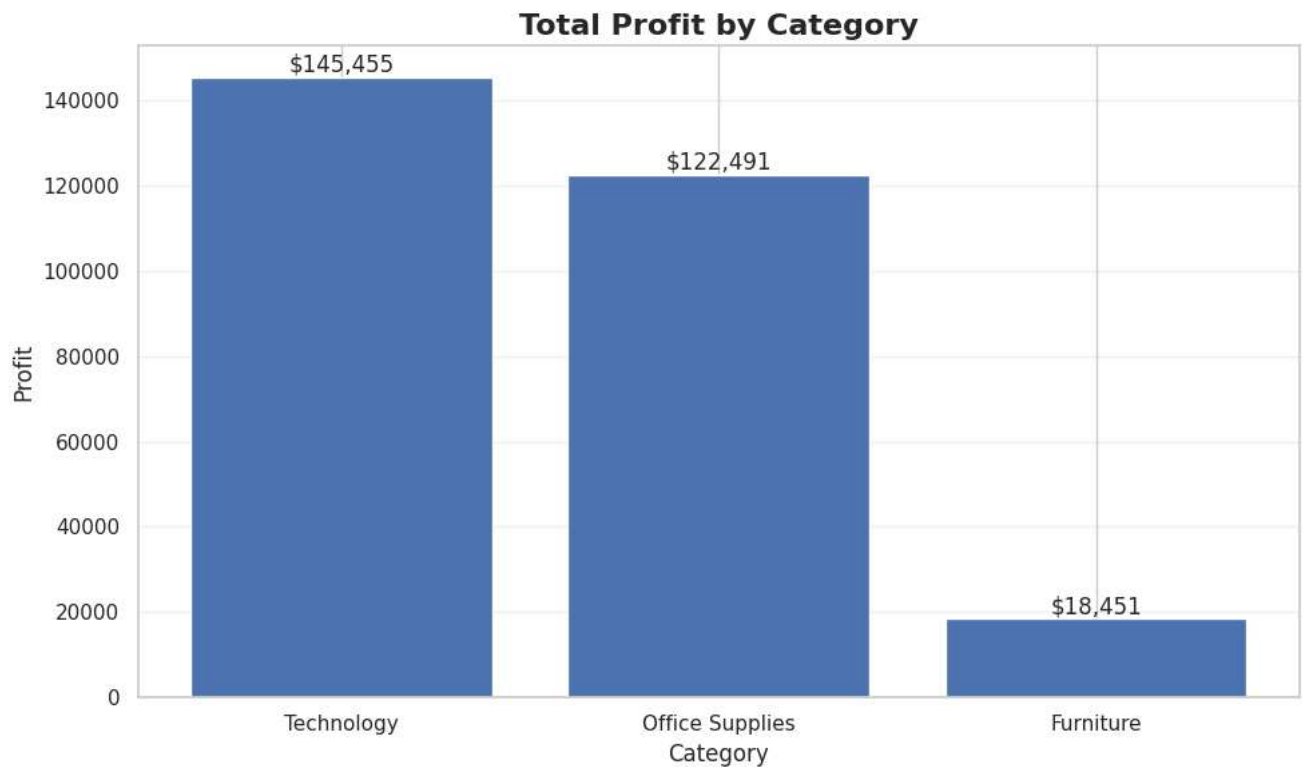
print(f"Total Sales      : ${total_sales:,.2f}")
print(f"Total Profit       : ${total_profit:,.2f}")
print(f"Total Quantity     : {total_quantity:,}")
print(f"Total Orders        : {total_orders:,}")
print(f"Average Sale       : ${average_sales:,.2f}")

print("=" * 45)
```

```
=====
      BUSINESS PERFORMANCE
=====
Total Sales      : $2,297,200.86
Total Profit     : $286,397.02
Total Quantity   : 37,873
Total Orders     : 5,009
Average Sale     : $229.86
=====
```

✓ Profit Analysis by Category

```
# =====  
# PROFIT BY CATEGORY  
# =====  
  
category_profit = (  
    df.groupby("Category")["Profit"]  
      .sum()  
      .sort_values(ascending=False)  
)  
  
plt.figure(figsize=(10, 6))  
  
bars = plt.bar(  
    category_profit.index,  
    category_profit.values  
)  
  
plt.title(  
    "Total Profit by Category",  
    fontsize=16,  
    fontweight="bold"  
)  
  
plt.xlabel("Category")  
plt.ylabel("Profit")  
  
plt.grid(axis="y", alpha=0.3)  
  
for bar in bars:  
    height = bar.get_height()  
    plt.text(  
        bar.get_x() + bar.get_width()/2,  
        height,  
        f"${height:,.0f}",  
        ha="center",  
        va="bottom"  
    )  
  
plt.tight_layout()  
plt.show()
```

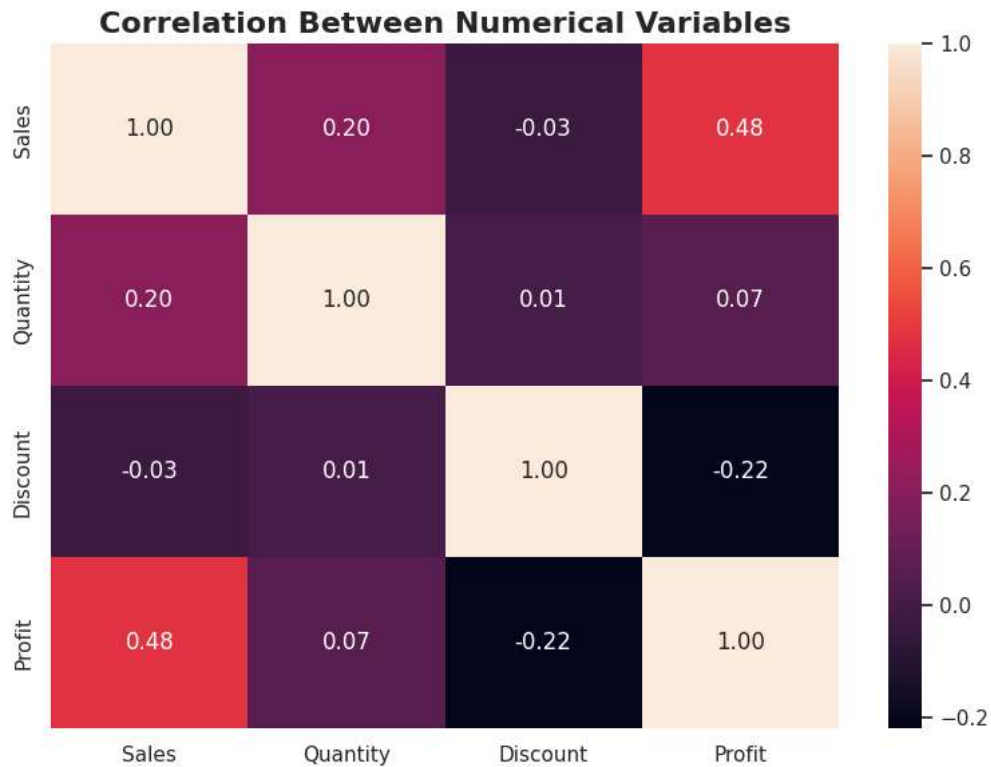


```
# =====  
# CORRELATION ANALYSIS  
# =====
```

```
correlation = df[[  
    "Sales",  
    "Quantity",  
    "Discount",  
    "Profit"  
]].corr()  
  
print(correlation)
```

	Sales	Quantity	Discount	Profit
Sales	1.000000	0.200795	-0.028190	0.479064
Quantity	0.200795	1.000000	0.008623	0.066253
Discount	-0.028190	0.008623	1.000000	-0.219487
Profit	0.479064	0.066253	-0.219487	1.000000

```
plt.figure(figsize=(8, 6))  
  
sns.heatmap(  
    correlation,  
    annot=True,  
    fmt=".2f"  
)  
  
plt.title(  
    "Correlation Between Numerical Variables",  
    fontsize=16,  
    fontweight="bold"  
)  
  
plt.tight_layout()  
plt.show()
```



Interactive Dashboard

An interactive dashboard is created using Plotly to allow users to explore sales performance across categories and regions.

```
# =====
# INTERACTIVE SALES DASHBOARD
# =====

fig = px.scatter(
    df,
    x="Sales",
    y="Profit",
    color="Category",
    size="Quantity",
    hover_data=[
        "Region",
        "Sub-Category",
        "Product Name"
    ],
    title="Interactive Sales vs Profit Dashboard"
)

fig.update_layout(
    width=1000,
    height=600
)

fig.show()
```

Interactive Sales vs Profit Dashboard



```
# =====  
# INTERACTIVE SALES BY REGION  
# =====  
  
fig = px.bar(  
    region_sales.reset_index(),  
    x="Region",  
    y="Sales",  
    color="Region",  
    title="Interactive Sales by Region"  
)  
  
fig.update_layout(  
    width=900,  
    height=500  
)  
  
fig.show()
```

